

Storage Wars

The pooler nobody patches, and Valkey learns to save without splitting in two

Issue #3

Wed, 07 Oct 2026

~39 min read

Level 3/5

Today's two stories

1. Your Postgres connection pooler is part of your attack surface, and it just had a rough fortnight.

PgBouncer 1.26.0 fixes three denial-of-service bugs, two of them reachable without a password, and Pgpool-II's 1 October releases fix seven more, including a certificate-login bypass and a way to take over its failover cluster. We explain each bug from the wire protocol up, and why "upgrade the pooler" is harder than it sounds this time.

2. Valkey 9.2's first release candidate can take a snapshot without calling `fork()`.

Since the early days of Redis, Redis-style databases have saved to disk by cloning themselves, which is why people run caches with a quarter or more of their RAM sitting idle. We look at how the new forkless save works inside, what it costs, and what's still missing before the 15 November GA.

3. On the radar: 6 more things that happened, listed at the end.

STORY 1 The pooler nobody patches: ten CVEs in PgBouncer and Pgpool-II

PGBOUNCER 1.26.0 (23 SEP 2026) · PGPOOL-II 4.7.3 AND BACK-BRANCHES (1 OCT 2026) · ~15 MIN READ

Ask a team how they patch Postgres and you'll usually get a good answer: a maintenance window, a minor-version upgrade, a failover. Ask how they patch the thing on port 6432 that every one of their application servers actually connects to, and the room tends to go quiet. Connection poolers are the most-connected and least-thought-about process in a typical Postgres stack. In the last two weeks the two most popular ones have shipped fixes for ten security vulnerabilities between them, and some of those bugs can be triggered by anyone who can open a TCP connection to the pooler, without knowing a single password.

WHAT HAPPENED

On **23 September 2026**, the PgBouncer project released **PgBouncer 1.26.0**, which "fixes three CVEs".¹ All three are denial-of-service bugs:^{1,2}

- **CVE-2026-19888**: a crash "triggerable by unauthenticated clients", caused by "a SCRAM client-final-message without a nonce".
- **CVE-2026-6668**: an infinite loop, also "triggerable by unauthenticated clients", caused by "an integer overflow in the packet buffer growth logic".

- **CVE-2026-6669**: "unbounded work during login, triggerable by a malicious PostgreSQL server", caused by an unbounded SCRAM iteration count. 1.26.0 now caps that count at 1,000,000.

The PostgreSQL community's apt repository published `pgbouncer 1.26.0-1.pgdg+1` packages for Debian and Ubuntu on 28 September.³

A week later, on **1 October 2026**, the Pgpool Global Development Group released **Pgpool-II 4.7.3, 4.6.8, 4.5.13, 4.4.18 and 4.3.21**, one fix release for every supported branch, with seven CVEs between them (CVE-2026-92867 to CVE-2026-92873).⁴ Six are in the **watchdog** *≈ Pgpool-II's built-in subsystem that lets several Pgpool-II servers monitor each other, elect a leader and coordinate failover* or its heartbeat process, and one is in certificate authentication. The CVE records, published on 30 September with JPCERT as the issuing authority, list every release from the 3.5 era up to 4.7.2 as affected.^{5,7}

This isn't PgBouncer's first security release this year. Its changelog shows 1.25.1 (3 December 2025) fixing CVE-2025-12819, and 1.25.2 (8 May 2026) fixing four more (CVE-2026-6664 to 6667).² That's three security releases in under ten months, for a component a lot of teams install once and forget about.

THE BACKGROUND YOU NEED

Why Postgres needs a pooler at all

PostgreSQL uses what its own documentation calls a **"process per user" model** *≈ PostgreSQL's design in which every client connection gets its own dedicated operating-system process on the server* : "every client process connects to exactly one backend process", and a supervisor process "spawns a new backend process every time a connection is requested".¹¹ Processes are heavyweight. Starting one costs time, and each one holds memory for its whole life, even while it's idle. A fleet of 200 application pods with a 20-connection pool each would ask Postgres for 4,000 processes, most of them doing nothing.

A **connection pooler** *≈ a proxy that sits between applications and the database, accepting many client connections and sharing a much smaller number of real database connections between them* fixes that. Applications connect to the pooler as if it were Postgres, and the pooler hands out a small set of real server connections as needed. Two poolers dominate self-managed setups.

PgBouncer is a small, focused proxy. By default it listens on port **6432**,¹² and you choose how aggressively it shares connections with `pool_mode` :¹²

- `session` : "Server is released back to pool after client disconnects. Default."
- `transaction` : "Server is released back to pool after transaction finishes."
- `statement` : "Server is released back to pool after query finishes."

The detail that matters most for today's story is architectural. PgBouncer is one single-threaded **event loop** *≈ a program design where one thread handles many connections by reacting to whichever one has data ready, instead of giving each connection its own thread* . As ClickHouse's engineers put it when they scaled it for their managed Postgres service, "A single process uses one CPU core, no matter how many the machine has."¹⁴ That's why it's so efficient, and it's also why a bug that crashes or hangs that one process takes **every** client connection down with it.

Pgpool-II is a bigger piece of **middleware** *≈ software that sits between applications and a database and adds features such as pooling, load balancing or failover*. Besides pooling, it can load-balance reads across replicas and orchestrate failover. To avoid becoming a single point of failure itself, you can run several Pgpool-II nodes and let the watchdog coordinate them. The watchdog nodes talk to each other over TCP (port 9000 by default) and send heartbeats over UDP (port 9694 by default). They can also hold a **virtual IP (VIP)** *≈ a floating IP address that moves to whichever server is currently the leader, so clients always connect to the same address* that applications connect to.¹⁵ Watchdog messages are protected by a shared secret, `wd_authkey`: "All Pgpool-II must have the same key", and leaving it empty "disables the watchdog authentication".¹⁵

How a Postgres login works: SCRAM in four messages

Two of PgBouncer's three bugs live in the login handshake, so it's worth seeing what that handshake actually looks like.

Modern Postgres authenticates passwords with **SCRAM-SHA-256** *≈ a challenge-response login method where client and server prove they both know the password without ever sending it, defined in RFC 5802 and RFC 7677*. PostgreSQL's protocol docs describe the exchange as a fixed sequence: the server offers mechanisms, the client sends a `client-first-message`, the server answers with a `server-first-message`, the client sends a `client-final-message`, and the server confirms with a `server-final-message` followed by `AuthenticationOk`.¹⁶

The SCRAM specification, RFC 5802, gives a worked example of what's inside those messages:¹⁷

```
C: n,,n=user,r=fyko+d2lbbFg0NRv9qkxdawL
S: r=fyko+d2lbbFg0NRv9qkxdawL3rfcNHYJY1ZVvWVs7j,
   s=QSXCR+Q6sek8bf92,i=4096
C: c=biws,r=fyko+d2lbbFg0NRv9qkxdawL3rfcNHYJY1ZVvWVs7j,
   p=v0X8v3Bz2T0CJGbJQyF0X+HI4Ts=
S: v=rmF9pqV8S7suAoZWja4dJRkFsKQ=
```

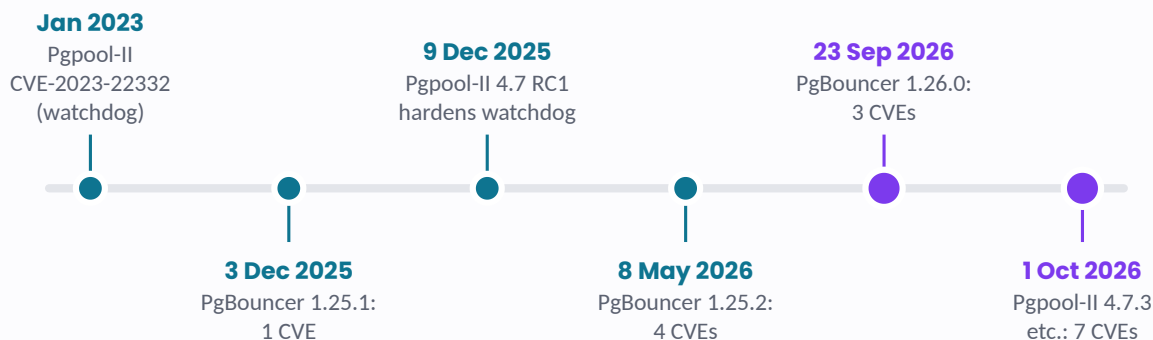
Three attributes matter here:¹⁷

- `r` is the **nonce** *≈ a random, use-once value that makes each login unique so a recorded exchange can't be replayed*. The client sends a random string, the server appends its own, and the client must echo the combined value back in its final message.
- `s` is the user's **salt** *≈ a random value stored with a password hash so that two users with the same password get different hashes*.
- `i` is the **iteration count** *≈ how many times the password is run through the hash function to derive the key; higher means slower to brute-force but also slower to log in*. The RFC says servers should announce "at least 4096".

The key is derived by a function called `Hi()`, which the RFC describes as "essentially, PBKDF2 with HMAC() as the pseudorandom function". In other words, the client hashes the password `i` times in a row. That's deliberate: it makes stolen password hashes expensive to crack. PostgreSQL 16 made the count configurable with the `scram_iterations` setting, which defaults to 4096.¹⁸

There's one twist that matters for CVE-2026-6669. A pooler plays **both** roles. Towards your application it's the SCRAM server, and towards Postgres it's a SCRAM **client**, logging in on your application's behalf. So PgBouncer receives an `i=` value from the database server and dutifully runs that many hash rounds.

A year of security releases for Postgres poolers



From the PgBouncer changelog, JVN on CVE-2023-22332, Pgpool-II 4.7 RC1 news and the Pgpool-II 4.7.3 announcement.

HOW IT WORKS

Let's take the bugs one at a time, starting with the ones anybody can reach.

CVE-2026-19888: the missing nonce that crashes PgBouncer

In a normal login, the client's final message carries the combined nonce (`r=...`) and its proof (`p=...`). The bug is in how PgBouncer parsed that message. Per the advisory summarised by Vulert, a malformed message could make "the parser incorrectly report success while leaving a required value unset".¹⁹ PgBouncer's own description is shorter: a crash caused by "a SCRAM client-final-message without a nonce".¹

Put those together and you get a classic **NULL pointer dereference** *≈ when a C program tries to read memory through a pointer that was never set, which makes the operating system kill the process*. The parser said "all good", the code that came next tried to compare the nonce it expected with one that was never stored, and the process died.

Two details make this serious. First, the attacker only needs to reach the port and start a SCRAM login: no valid username or password is required, because the crash happens **during** authentication. Second, because PgBouncer is a single process, the crash drops every client connection at once, not just the attacker's.^{19,14}

CVE-2026-6668: a buffer that never stops growing

Every network server has to decide how much memory to allocate for an incoming packet. A common approach is to start with a small buffer and grow it, often by doubling, until the packet fits. PgBouncer's bug was "an integer overflow in the packet buffer growth logic", which could send the process into an infinite loop, triggerable by unauthenticated clients.¹

An **integer overflow** *≈ when arithmetic produces a number too big for the variable holding it, so it wraps around to a small or negative value* is easiest to understand with a toy example. The code below is **illustrative**, not PgBouncer's:

```
/* illustrative only */
uint32_t size = 4096;
while (size < needed)    /* needed is close to 4 GiB */
    size *= 2;           /* 2^31 * 2 wraps to 0, and 0 * 2 is still 0 */
```

Once the doubling passes the largest value a 32-bit number can hold, it wraps to zero and stays there, and `size < needed` is true forever. The real fix is in PgBouncer's code, but the effect matches the advisory. An attacker who announces a large enough packet leaves PgBouncer spinning at 100% of its single core, and since that core is the event loop, "all pooled database connections" stall until someone kills the process.²⁰

CVE-2026-6669: when the database attacks the pooler

The third bug points the other way. When PgBouncer logs in to a Postgres server with SCRAM, the server's `server-first-message` tells it how many times to hash (`i=`). Before 1.26.0 PgBouncer accepted any value. A "malicious PostgreSQL server" could therefore announce a huge count and leave PgBouncer computing HMACs for a very long time.¹ 1.26.0 caps the count at 1,000,000.²

How much does the iteration count matter? Michael Paquier, a PostgreSQL committer, measured it when `scram_iterations` was added. On his test machine, connections per second against Postgres fell from about 236 at the default 4096 iterations to about 0.16 at 10 million.¹⁸

Login throughput vs SCRAM iteration count (one committer's test)



Measured by Michael Paquier on PostgreSQL 16 (source). It shows how the cost scales, not PgBouncer's own numbers. A malicious server can pick any count, which is why 1.26.0 caps it at 1,000,000.

"A malicious Postgres server" sounds far-fetched, until you remember the cases where the pooler doesn't fully trust the server: a multi-tenant platform pooling connections to databases its customers control, a compromised replica, or traffic that an attacker can redirect. That's also why this one is the least urgent of the three for a typical single-team setup.

Where each PgBouncer 1.26.0 fix sits

ANYONE ON THE NETWORK

PGBOUNCER (1 PROCESS)

POSTGRESQL

Bad SCRAM login

no nonce: crash (19888)

Huge packet

overflow: loop (6668)

Event loop
all clients share it

Hostile server
huge i=: CPU (6669)

Pgpool-II: the watchdog was the soft spot

Pgpool-II's seven CVEs cluster around one feature. Here's what the 1 October announcement says each one does, with the CVSS v4.0 scores from the CVE records where we could open them:^{4,5,6,7,8,9}

- **CVE-2026-92867 (8.7):** malformed watchdog messages during failover let an attacker "write an arbitrary 32-bit value to an arbitrary memory address". The CVE record says it requires an authenticated attacker and can lead to "arbitrary code execution".
- **CVE-2026-92869 (7.1):** malformed watchdog messages overwrite memory "beyond the boundaries of fixed-size arrays", crashing the process.
- **CVE-2026-92870:** failover messages with "arbitrary-length data" corrupt the stack and crash a Pgpool-II process.
- **CVE-2026-92871 (8.7):** a NULL pointer dereference in watchdog node-to-node authentication. When an auth key is configured, messages that simply omit the authentication information aren't handled correctly. The CVE record rates it as needing no authentication.
- **CVE-2026-92872:** an information leak in the heartbeat receiver process.
- **CVE-2026-92873 (6.9):** a flaw in watchdog promotion that lets an attacker "bypass authentication key checks and promote a watchdog node of their choice to leader".
- **CVE-2026-92868 (6.9):** the odd one out, in **certificate authentication**. Pgpool-II "does not properly handle NUL bytes (\0)" in the Common Name of a client certificate, so "a malicious client" can "connect to the Pgpool-II server as another user without a password".

That last one is worth a closer look, because it's a classic C bug. In C, strings end at the first **NUL byte** ≈ a zero byte, which C uses to mark the end of a string. A certificate's **Common Name (CN)** ≈ the field in an X.509 certificate that names its owner; with certificate authentication, the CN is matched against the database user name is a length-prefixed value, though, so it can legally contain a zero byte in the middle. If one part of the code compares the CN as a C string, it stops reading at the NUL. Illustratively, a certificate whose CN is `alice\0.attacker.example` could be read as just `alice` by a C-string comparison, while whoever issued the certificate saw the full name. The CVE record classifies it as improper certificate validation (CWE-295).⁶

The watchdog bugs matter because of what the watchdog controls. It decides which Pgpool-II node is leader, and therefore who holds the virtual IP your applications connect to and who drives failover.¹⁵ A bug that lets an attacker pick the leader (92873) or crash the watchdog (92871) is a bug in your high-availability layer, the part you installed precisely so that things wouldn't fall over.

None of this came out of nowhere. In January 2023, CVE-2023-22332 disclosed a watchdog information leak in which "a specific database user's authentication information may be obtained by another database user".²¹ And the Pgpool-II 4.7 RC1 release in December 2025 already listed "enhanced protections" for "watchdog and heartbeat receiver components".²² This batch looks like the continuation of that hardening work, now reaching every supported branch.

WHAT IT MEANS FOR YOU

Find out what you're actually running. For PgBouncer, connect to its admin console and run `SHOW VERSION;`, which shows "the PgBouncer version string", or run `pgbouncer -V` on the host.¹³ Anything up to and including 1.25.2 is affected by all three new CVEs.¹⁹ For Pgpool-II you need 4.7.3, 4.6.8, 4.5.13, 4.4.18 or 4.3.21 depending on your branch; earlier releases on each branch are affected.^{4,5}

Don't assume your distribution has caught up. When we checked Debian's security tracker for CVE-2026-92873, `pgpool2` in Debian 12 "bookworm" (4.3.5-1+deb12u1) and Debian 13 "trixie" (4.6.1-2) were still marked vulnerable, with only unstable (4.7.3-1) fixed.¹⁰ For PgBouncer, the PGDG apt repository had 1.26.0 within five days.³ If you install poolers from your OS's default repositories, that gap is your exposure window.

Don't assume your cloud provider has caught up either. Managed poolers are fine, but they follow their own schedule. Microsoft's documentation for the built-in PgBouncer in Azure Database for PostgreSQL said, when we opened it on 7 October, that the version deployed "is **1.25.2**", and that "Azure manages updates of PgBouncer".²³ (A provider can patch without changing the version number, so treat this as a reason to ask, not as proof of exposure.) For a sense of the usual lag, 1.25.1 shipped upstream in December 2025² and an Azure update listing announced it as generally available on the service in April 2026.²⁴

Plan the PgBouncer restart, because 1.26.0 changed how restarts work. This release removed the "online restart" feature, the `-R` flag that let a new PgBouncer process take over the old one's open sockets, together with the `SHOW FDS` and `SUSPEND` admin commands. Restarting now means stop and start, "which disconnects clients".^{3,2} The supported way to upgrade without a blip is to run several PgBouncer processes on the same port with `so_reuseport` and restart them one at a time, using `SHUTDOWN WAIT_FOR_CLIENTS`, which makes a process "stop accepting new connections and shutdown the process once all existing clients have disconnected".^{13,12}

Upgrading PgBouncer: 1.25 vs 1.26

≤ 1.25: hand over the sockets

```
# start the new binary; it takes
# over the old process's sockets
pgbouncer -R /etc/pgbouncer.ini
# clients stay connected
```

1.26+: rolling restart

```
# pgbouncer.ini on every process
so_reuseport = 1
# then, one process at a time:
#   SHUTDOWN WAIT_FOR_CLIENTS;
#   start the 1.26 binary
```

From the 1.26.0 package notes and the usage docs. Running several processes also gets around the one-core limit.

ClickHouse's managed Postgres team took exactly this multi-process approach to get past the one-core ceiling. On a 16-vCPU instance, a fleet of 16 PgBouncer processes reached about 336k transactions per second, against about 87k at best for a single process. That's the company's own benchmark.¹⁴ So the restart change pushes you towards a setup that also scales better. The catch they ran into is that query-cancel requests can land on a different process from the one holding the session, which they solved by having processes forward cancellations to each other.¹⁴

Shrink who can reach the pooler. Two of the PgBouncer bugs and at least one of the Pgpool-II bugs need nothing more than network access. The interim advice in the advisories is the same everywhere: restrict connections to trusted clients at the network level until you've upgraded.^{19,6} Pgpool-II's watchdog (TCP 9000) and heartbeat (UDP 9694) ports should only ever be reachable from the other Pgpool-II nodes,¹⁵ and `wd_authkey` should never be empty.

Watch out for

- **"It's only DoS" undersells it.** For a pooler, a crash is an outage for every service behind it, and an infinite loop is worse, because the process stays up and health checks that only test "is the port open" may not notice.
- **Other 1.26.0 behaviour changes.** PgBouncer now tracks `search_path` and `default_transaction_read_only` by default, adds a `pool_idle_timeout` setting (default 0, disabled), allows `query_wait_timeout` per user and database, raises the maximum password length to 65,535 bytes and needs a C11 compiler to build.^{1,2,12} Read the changelog before you roll it out, especially if you build from source.
- **Certificate auth isn't automatically safer.** CVE-2026-92868 is a reminder that "no password" logins move the risk into certificate parsing. If you use `cert` authentication on Pgpool-II, this is the CVE to patch first.
- **Exploitation status.** The CVE records we opened don't list these in CISA's Known Exploited Vulnerabilities catalogue, and their EPSS scores are low.^{5,8} That's a reason to patch calmly, not a reason to skip it.

WHAT HAPPENS NEXT

PgBouncer 1.26.0 is the current release, and its changelog gives no sign of a 1.25.x backport, so 1.26.0 is the upgrade target.² For Pgpool-II, every supported branch already has a fix,⁴ so the open question is how quickly distributions pick them up. If you're on Debian stable, keep an eye on the security tracker.¹⁰ The bigger lesson is a process one: put the pooler on the same patch calendar as the database. It's smaller than Postgres, but it sees every connection first.

Words from this story

Watchdog: Pgpool-II's subsystem that lets several Pgpool-II servers monitor each other, elect a leader and coordinate failover.

"Process per user" model: PostgreSQL's design in which every client connection gets its own operating-system process on the server.

Connection pooler: a proxy that accepts many client connections and shares a much smaller number of real database connections between them.

Event loop: one thread serving many connections by reacting to whichever one has data ready.

Middleware: software between applications and a database that adds features such as pooling, load balancing or failover.

Virtual IP (VIP): a floating IP address that moves to whichever server is currently the leader.

SCRAM-SHA-256: a challenge-response login method in which both sides prove they know the password without sending it.

Nonce: a random, use-once value that makes each login unique.

Salt: a random value stored with a password hash so that identical passwords hash differently.

Iteration count: how many times the password is hashed to derive the key; slower to crack, slower to log in.

NULL pointer dereference: reading memory through a pointer that was never set, which crashes the process.

Integer overflow: arithmetic that exceeds what a variable can hold and wraps around to a small value.

NUL byte: a zero byte, which C uses to mark the end of a string.

Common Name (CN): the field in an X.509 certificate that names its owner, matched against the user name in certificate authentication.

Valkey 9.2 learns to save without splitting in two

VALKEY 9.2.0-RC1 · RELEASED 16 SEP 2026 · GA TARGETED FOR 15 NOV 2026 · ~14 MIN READ

If you run Redis or Valkey on a managed service, there's a good chance a quarter of the memory you pay for sits empty on purpose. It isn't waste in the usual sense. It's insurance against one moment: the instant the server saves a snapshot to disk by cloning itself in two. Since the early days of Redis that trick has been both the cleverest and the most hated part of Redis-style persistence. Valkey 9.2's first release candidate ships an alternative that doesn't clone anything, and the way it keeps the snapshot consistent is a neat piece of engineering.

WHAT HAPPENED

Valkey **9.2.0-rc1** was released on **16 September 2026**, marked "Upgrade Urgency: LOW" because it is "the first release candidate of Valkey 9.2.0".²⁵ The project's release plan targets **general availability on 15 November 2026**, and it named three "Tier 0" flagship features that were allowed to block the release: a B+ tree replacement for sorted sets, **Forkless Save**, and streaming compression.²⁶

The release notes describe the headline feature as **opt-in forkless RDB snapshots**, controlled by two new settings, `forkless-infrastructure-enabled` and `bgsave-default-method`, plus new `INFO persistence` fields that report the save method and its progress.²⁵ The same release candidate brings a long list of other changes:²⁵

- **ACL roles:** named, reusable permission sets managed with `ACL SETROLE`, `DELROLE`, `GETROLE` and `ROLES`, plus an `ACL DIGEST` command that "returns a fingerprint of the ACL rules currently in effect".
- **Path Hash:** a new radix-tree-backed data type with `PH*` commands for exact lookup, longest-prefix matching and prefix traversal.
- **Server-side hot key detection** (`HOTKEYS GET` / `HOTKEYS RESET`), optional **replication throttling** that slows client writes when replicas fall behind, `INCREX` (increment with expiry), and new stream commands `XACKDEL` and `XDELEX`.
- **Performance:** large sorted sets now use a B+ tree instead of a skiplist, LZ4 compression for RDB files, and batch processing for `HMGET`, `SMISMEMBER` and `ZMSCORE`.
- **One behaviour change to note:** active key expiration now increments the dirty counter, which means automatic `BGSAVE` can trigger more often.

We'll spend this story on forkless save, because it changes the most basic operational assumption about running Valkey.

THE BACKGROUND YOU NEED

Valkey, one thread, and two ways to persist

Valkey *≈ an open-source fork of Redis that keeps its data model and commands* is an in-memory key-value store. Its commands run on a single main thread. AWS's ElastiCache docs remind readers that "these engines are single

threaded".³³ Everything lives in RAM, so if you want your data to survive a restart, the server has to write it out somewhere.

There are two ways to do that. An **RDB snapshot** *≈ a compact binary file containing the whole dataset as it was at one moment* captures everything at once. You trigger one with `BGSAVE` or let rules such as `save 60 1000` do it ("every 60 seconds if at least 1000 keys have changed").³¹ An **AOF (append-only file)** *≈ a log of every write command, replayed on startup to rebuild the dataset* records changes as they happen and periodically gets compacted by an AOF rewrite. Replicas joining a primary also need a full copy of the data to start from, called a **full sync** *≈ the initial copy of the entire dataset that a primary sends to a new or badly lagging replica*, which is normally an RDB produced on the fly.

The fork trick, and its bill

How do you write a consistent picture of a dataset that's changing thousands of times a second, without stopping the single thread that changes it? Redis's answer, which Valkey inherited, is to ask the operating system for help. The persistence docs describe it plainly: "Valkey forks. We now have a child and a parent process. The child starts to write the dataset to a temporary RDB file. When the child is done writing the new RDB file, it replaces the old one."³¹

fork() *≈ the Unix system call that creates a new process as an exact copy of the current one* looks expensive but usually isn't, thanks to **copy-on-write (COW)** *≈ an OS trick where parent and child share the same physical memory pages until one of them writes to a page, at which point that page is copied*. Right after the fork, parent and child share every page of memory. The child sees a frozen picture of the data as it was at the moment of the fork, which is exactly what a snapshot needs. Meanwhile the parent keeps serving clients. Only when the parent **writes** to a page does the kernel copy that page, so the child's view stays untouched.

It's elegant, and it has three costs.

1. The fork itself pauses the server. The kernel has to copy the process's **page table** *≈ the operating system's map from a process's virtual memory addresses to physical memory, with one entry per page*. Valkey's latency docs work through the numbers: with 4 kB pages, "a large 24 GB Valkey instance requires a page table of $24\text{ GB} / 4\text{ kB} * 8 = 48\text{ MB}$ ", and copying it takes time. You can see how long your last fork took in `INFO` as `latest_fork_usec`.³² The forkless design proposal says fork times "exceeding 100ms can often be seen".²⁷

2. Memory can nearly double. Every page the parent writes during the save gets copied. On a write-heavy instance that can be most of them: the design proposal warns that copy-on-write "may require duplicating all memory pages, creating potential 2x memory consumption".²⁷ It gets worse with **transparent huge pages (THP)** *≈ a Linux feature that silently groups memory into 2 MB pages instead of 4 kB ones*. Valkey's docs explain that a few event-loop runs touching a few thousand pages can cause "the copy on write of almost the whole process memory", which is why they tell you to disable THP.³²

3. You pay for that headroom all the time. Because nobody wants the kernel's out-of-memory killer to arrive halfway through a snapshot, operators keep spare RAM around permanently. AWS's ElastiCache guidance is explicit: "we recommend that you set `reserved-memory-percent` to 25 (25 percent) for Valkey and Redis

OSS."³³ The Valkey maintainers' planning issue for the 9.2 blog post describes the same habit from the other side: users provisioning hosts "with 2x memory margins" to absorb copy-on-write.³⁰

The road to forkless persistence in Valkey 9.2



From the 9.2.0-rc1 release notes, the 9.2 release plan, PR #4797, the valkey-agents audit issues and a DEV Community benchmark.

HOW IT WORKS

The design is laid out in Valkey issue #2878, "Valkey forkless save, sync, slot-migration".²⁷ The snapshot code landed as PR #4460, which the follow-up full-sync PR names as its foundation.²⁸ The core idea is to replace "the kernel keeps a frozen copy for us" with "we walk the keyspace ourselves, and we make sure nobody changes a key before we've written it out".

A background iterator, and the main thread keeps working

The proposal introduces a component called **bgiteration**, "a decoupled utility that iterates over key-value pairs asynchronously without forking".²⁷ A save runs in four stages:²⁷

1. The RDB **header** is written on the main thread.
2. **bgiteration** walks every key and hands key-value pairs to a background thread.
3. That thread, which the design calls **Threadsave**, serializes each pair into RDB format and writes it to the file.
4. Final callbacks on the main thread write the end marker and close the file.

Because the main thread is only handing out work, it keeps executing client commands throughout. The proposal notes that the output is a normal RDB file, "fully compatible", although "key ordering may differ" from a fork-based save.²⁷

Keeping the picture consistent: a 4-byte epoch per key

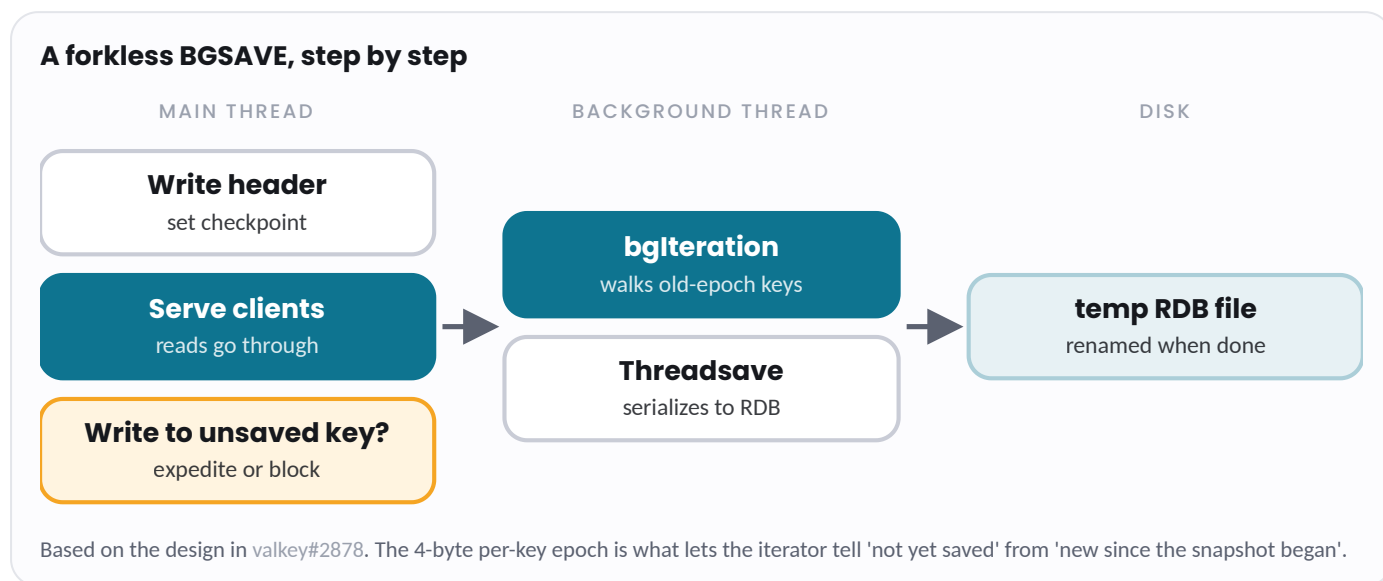
Here's the hard part. With `fork()`, consistency is free, because the child sees memory exactly as it was at one instant. Without a fork, the iterator is walking a live dataset. If a client updates key A before the iterator reaches it, the snapshot would contain the **new** value of A next to **old** values of keys written earlier, and it would no longer be a picture of any single moment.

The design solves this with a per-key **epoch** *≈ a small counter stored with each key, recording when it was last created or modified relative to the snapshot's starting point*. Every key carries a 4-byte epoch field, "updated at creation and modification".²⁷ When a consistent save starts, the server records a consistency checkpoint. From then on:²⁷

- Keys whose epoch is **older** than the checkpoint haven't been touched since the save began, so the iterator serializes them as they are.
- Keys with a **newer** epoch were either created after the snapshot started, so they don't belong in it, or have already been processed.
- When a write command targets a key the iterator **hasn't reached yet**, the server can't let the write go ahead blindly, because that would destroy the value the snapshot needs. It either processes that key immediately, out of order, or blocks the client briefly. For small keys it can simply clone the value instead.

Reads aren't affected: "Most read operations proceed unaffected," as the proposal puts it.²⁷ Elestio's walkthrough of the RC describes the write path the same way. The client is blocked while the key is "expedited to the front of the background thread's work list", then unblocked, and its command re-executes.³⁴

If that sounds familiar, it's the same idea as copy-on-write, moved from the kernel's 4 kB (or 2 MB) pages into the database's own keys. Instead of copying a whole page because one byte on it changed, Valkey saves the one key that's about to change, and only if the snapshot still needs it.



Why 4 bytes per key is the real price

The epoch is why you have to switch the feature on at startup. Adding 4 bytes to every key is a memory-layout decision, and the server can't retrofit it onto keys already in RAM. So there are two settings:^{29,34}

- `forkless-infrastructure-enabled yes` must be set before the server starts. It reserves the per-key epoch.
- `bgsave-default-method forkless` picks the save method and can be changed at runtime with `CONFIG SET`. If you try it on a server started without the infrastructure flag, `CONFIG SET` fails with an error saying forkless "requires the server to be started with forkless-infrastructure-enabled yes".²⁹

The default stays `fork`, for backward compatibility.³⁴

valkey.conf: classic vs forkless snapshots

9.1 and default 9.2

```
save 60 1000
# BGSAVE forks a child;
# keep ~25% RAM spare
# for copy-on-write
```

9.2 forkless (opt-in)

```
save 60 1000
# startup-only: +4 bytes/key
forkless-infrastructure-enabled yes
# can be changed with CONFIG SET
bgsave-default-method forkless
```

Setting names from the 9.2.0-rc1 release notes and PR #4602. The 25% figure is AWS's ElastiCache recommendation for fork-based saves.

The overhead is easy to estimate: 4 bytes times your key count. Elestio works it out as 2 GB at 500 million keys.³⁴ For a dataset of a few million keys, it's a rounding error.

What the trade looks like in practice

Valkey's own design document is honest that this isn't a free lunch. Its comparison table lists the forkless advantages as "minimal additional memory overhead", no "~100-300ms fork latency" and no client-output-buffer growth. It also says fork keeps real strengths: "20-25% faster end-to-end execution", better handling of very large keys, simpler maintenance, and better handling of large module metadata.²⁷

fork() + copy-on-write

- Consistency comes free from the kernel
- Pause while the page table is copied
- Memory can approach 2x under heavy writes
- 20-25% faster end to end (design doc)
- Still used for AOF rewrite and replica sync in RC1

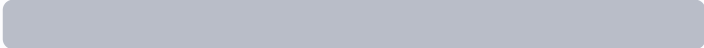
VS

Forkless (bgiteration)

- Background thread walks the keyspace
- No fork pause, no page copies
- Fixed cost: 4 bytes per key
- Writes to unsaved keys may block briefly
- Opt-in; restart needed to enable

The first public numbers come from a community test, not from the project, so treat them as one data point. A DEV Community author ran 9.2.0-rc1 in Docker with 3 million 300-byte keys (about 1.1 GB) under roughly 27,000 SET s per second from eight threads, and triggered BGSAVE in both modes, seven trials each.³⁵

One community test of 9.2.0-rc1: fork vs forkless BGSAVE

Memory spike: fork  354.4 MB

Memory spike: forkless  9.9 MB

Worst latency: fork  20.1 ms

Worst latency: forkless  7.2 ms

From a single DEV Community benchmark (3M keys, ~27k SET/s, 7 trials per mode, cgroup memory deltas, mean of worst-case latencies). The two metrics use different units and aren't comparable to each other. Not an official Valkey benchmark.

The same test reports the other side of the trade. The save took 5–6 seconds instead of 3, and write throughput dropped by about 15% while it ran.³⁵ The memory number is also roughly what the arithmetic predicts: 3 million keys times 4 bytes is 12 MB of epoch fields, and the measured spike was under 10 MB, against hundreds of megabytes of copied pages for the fork.

WHAT IT MEANS FOR YOU

Don't run 9.2.0-rc1 in production. Valkey's own guidance on release candidates is that "a release candidate is never intended for a production system".³⁶ What you can do now is test it in staging with your real write pattern, which is exactly what the RC is for.

Measure your fork tax first. Look at `latest_fork_usec` in `INFO` after a save,³² and at your peak memory during `BGSAVE` compared with steady state. If both are small, because your dataset is small or your writes are light, forkless won't change much. The community test found negligible differences on idle data,³⁵ which matches the theory: copy-on-write only costs anything when pages are written during the save.

Understand what still forks in RC1. Forkless covers `BGSAVE` snapshots. AOF rewrites still fork, according to the community test,³⁵ and Elestio notes that replica full sync still uses `fork()` in RC1.³⁴ A full-sync replacement is in progress: PR #4797, "Forkless Full-Sync", opened on 28 September and still open, "extends forkless save from 'snapshot to disk' to serving a replica full resynchronization", streaming the RDB straight to replicas with replication data sent inline, "so there is no client-output-buffer growth during the sync".²⁸ Until something like that ships, a primary that forks for a new replica can still hit the old memory spike.

If you use modules or big Lua scripts, test those paths. Elestio reports that modules must opt in with `VALKEYMODULE_OPTIONS_HANDLE_FORKLESS_SAVE`, and that Lua scripts that don't declare their keys block the main thread synchronously during a forkless save, because the server can't know in advance which keys they'll touch.³⁴

On managed services, watch the reserved-memory knob. If forkless becomes the norm, the 25% reserved-memory habit³³ is the first thing that could shrink, and that's money. AWS's ElastiCache page already says its Valkey and Redis OSS engines "support snapshots and synchronizations using a forkless save process", alongside the classic fork explanation.³³ What that means for open-source 9.2 on ElastiCache is something to ask AWS once GA lands, not something to assume.

Watch out for

- **Two known RC1 bugs in failure paths.** A Valkey maintainer's audit tracker has two open issues from 16 September. First, if a forkless `BGSAVE` can't create its temp file (a read-only disk, wrong permissions), it returns early without recording the error, so `rdb_last_bgsave_status` stays `ok` and `stop-writes-on-bgsave-error` "never engages".³⁷ Second, shutting down mid-save leaks the partial `temp-forkless-<pid>-<n>.rdb` file, one per restart, with no cleanup at startup.³⁸ Check whether they're fixed in a later RC before relying on alerts that read `rdb_last_bgsave_status`.
- **Saves are slower.** Expect longer snapshots and lower write throughput while one runs. Fork-based saves are 20–25% faster end to end, per the design doc.²⁷
- **The infrastructure flag needs a restart.** You can't turn on the per-key epoch on a running server.²⁹
- **More saves than before.** Separately from forkless, 9.2 counts active key expiry as a change, so `save` rules can fire more often on expiry-heavy workloads.²⁵

WHAT HAPPENS NEXT

The release plan targets **9.2.0 GA on 15 November 2026**,²⁶ and Valkey's release-candidate policy says that after the final RC "the delta between the final RC and GA" is "intended to be small".³⁶ The things to watch between now and then are whether forkless full sync (PR #4797) makes it into 9.2, and whether the audit bugs are fixed. A blog post by the full-sync author is planned "coordinated with" the 9.2 release, and its draft outline says it can cover snapshots only, with sync as "coming soon", if full sync misses the cut.³⁰ After that, the design document's third target, forkless **slot migration** for clusters, is the next obvious step.²⁷

Words from this story

Valkey: an open-source fork of Redis that keeps its data model and commands.

RDB snapshot: a compact binary file containing the whole dataset as it was at one moment.

AOF (append-only file): a log of every write command, replayed on startup to rebuild the data.

Full sync: the complete copy of the dataset a primary sends to a new or badly lagging replica.

fork(): the Unix system call that creates a new process as a copy of the current one.

Copy-on-write (COW): parent and child share memory pages until one writes, and then that page is copied.

Page table: the OS map from a process's virtual addresses to physical memory, one entry per page.

Transparent huge pages (THP): a Linux feature that groups memory into 2 MB pages, making copy-on-write much more expensive.

Epoch: a 4-byte counter on each key showing whether it changed after the snapshot started.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **Apache Kafka 4.2.2 (29 Sep):** a "bugfix release that includes several critical fixes" for the 4.2 line, while the Kafka blog still shows no 4.4.0 announcement as of 7 October.³⁹
- **Databricks Lakebase Search is GA (28 Sep):** BM25 full-text (`lakebase_text`) and vector (`lakebase_vector`) search inside Lakebase Postgres on AWS and Azure, with vendor claims of "97% recall at 71ms P99 latency" on a 100M-vector benchmark.⁴⁰
- **Snowflake hybrid tables (24 and 28 Sep):** `CHECK` constraints and inline stored procedures on hybrid tables are now generally available.⁴¹
- **Snowflake dynamic tables survive failover (30 Sep):** dynamic tables now keep refreshing incrementally after a failover, now GA together with Optimized Refresh and RPO Assurance for failover groups.⁴¹
- **LinkedIn on ClickHouse (1 Oct):** LinkedIn folded a three-service metric metadata system into one ClickHouse index that serves 150,000+ queries per minute across 13 billion+ metrics at a 68 ms average.⁴²
- **Databricks (2 Oct):** customer-managed keys for query history are now generally available.⁴³

1. **PgBouncer 1.26.0 released - Fixes three CVEs**, postgresql.org, 23 Sep 2026.
2. **PgBouncer changelog**, pgbouncer.org (entries for 1.25.1, 1.25.2 and 1.26.0).
3. **pgbouncer 1.26.0-1.pgdg+1 package update**, apt.postgresql.org, 28 Sep 2026.
4. **Pgpool-II 4.7.3, 4.6.8, 4.5.13, 4.4.18 and 4.3.21 released**, postgresql.org, 1 Oct 2026.
5. **CVE-2026-92867**, OpenCVE (CVE record, published 30 Sep 2026).
6. **CVE-2026-92868**, OpenCVE (CVE record, published 30 Sep 2026).
7. **CVE-2026-92869**, OpenCVE (CVE record, CNA JPCERT, published 30 Sep 2026).
8. **CVE-2026-92871**, OpenCVE (CVE record).
9. **CVE-2026-92873**, OpenCVE (CVE record, published 30 Sep 2026).
10. **CVE-2026-92873**, Debian security tracker, viewed 7 Oct 2026.
11. **How Connections Are Established**, PostgreSQL 17 documentation.
12. **PgBouncer configuration**, pgbouncer.org.
13. **PgBouncer usage**, pgbouncer.org.
14. **Scaling PgBouncer in ClickHouse Managed Postgres**, Kaushik Iska, ClickHouse, 1 Jul 2026.
15. **Watchdog configuration**, Pgpool-II 3.6 documentation (Crunchy Data mirror).
16. **SASL Authentication**, PostgreSQL 18 documentation.
17. **RFC 5802: Salted Challenge Response Authentication Mechanism (SCRAM)**, IETF, 2010.
18. **Postgres 16: scram_iterations**, Michael Paquier.
19. **CVE-2026-19888: PgBouncer NULL pointer dereference in SCRAM client-final-message parsing**, Vulert.
20. **CVE-2026-6668: PgBouncer integer overflow leading to denial of service**, Vulert.
21. **JVN#72418815: Pgpool-II information disclosure (CVE-2023-22332)**, JVN, 23 Jan 2023.
22. **Pgpool-II 4.7 RC1 released**, pgpool.net, 9 Dec 2025.
23. **PgBouncer in Azure Database for PostgreSQL**, Microsoft Learn, viewed 7 Oct 2026.
24. **PgBouncer 1.25.1 support generally available in Azure Database for PostgreSQL flexible server**, devicebase.net (Azure updates listing), Apr 2026.

25. [Valkey 9.2.0-rc1 release notes](#), mirrored from the GitHub release, 16 Sep 2026.
26. [Valkey 9.2 Release Plan \(#4218\)](#), valkey-io/valkey on GitHub.
27. [Valkey forkless save, sync, slot-migration \(#2878\)](#), valkey-io/valkey on GitHub.
28. [Forkless Full-Sync \(PR #4797\)](#), valkey-io/valkey on GitHub, opened 28 Sep 2026.
29. [Validate the forkless bgsave config pair in an apply function \(PR #4602\)](#), valkey-io/valkey on GitHub.
30. [Blog post: Forkless Save and Full Sync \(#683\)](#), valkey-io.github.io on GitHub.
31. [Persistence](#), Valkey documentation.
32. [Diagnosing latency issues](#), Valkey documentation.
33. [Ensuring you have enough memory to make a Valkey or Redis OSS snapshot](#), Amazon ElastiCache documentation.
34. [Valkey 9.2 Forkless Snapshots: BGSAVE Without fork\(\)](#), Elestio, 21 Sep 2026.
35. [Valkey 9.2's forkless BGSAVE cuts my memory spike from 350MB to 10MB](#), Alex Georgiev, DEV Community, 4 Oct 2026.
36. [On release candidates](#), Kyle Davis, Valkey blog, 28 Apr 2026.
37. [forkless-save: a forkless BGSAVE that cannot create its temp file leaves rdb_last_bgsave_status ok \(#78\)](#), madolson/valkey-agents on GitHub, 16 Sep 2026.
38. [forkless-save: shutdown during a forkless save leaks the partial temp RDB \(#73\)](#), madolson/valkey-agents on GitHub, 16 Sep 2026.
39. [Apache Kafka blog](#), Apache Software Foundation (4.2.2 announcement dated 29 Sep 2026).
40. [Lakebase Search: state-of-the-art full-text and vector search in Postgres](#), Databricks, 28 Sep 2026.
41. [All release notes](#), Snowflake documentation.
42. [How LinkedIn extended ClickHouse from distributed tracing to metric discovery and analytics](#), ClickHouse, 1 Oct 2026.
43. [Databricks release notes: October 2026](#), Databricks.